

COMP3222 Machine Learning Technologies

Lab Sheet 3 (Week 4)

When: Monday 12 October 2026, 13:00–15:00. **Based on:** Week 3 lectures, Regression and Ensembles. **Repository:** <https://github.com/TonyBagnall/COMP3222-2026>

This lab practices encoding categorical features, comparing regression models, and building a simple voting ensemble. Start with the files and run instructions in `code/lab3/README.md`. There are three tasks and an optional extension.

Task 1: Implement scikit-learn transformers

Open `code/lab3/lab3_task1.py`. Complete `frequency_encode(x)` so that each string category becomes the proportion of rows containing it. Complete `impact_encode(x, y)` so that each category becomes the proportion of class 1 among rows in that category. Here `y` contains only 0 and 1, so this is the category's mean label. See Week 3 slides 49 and 50. Call `check_functions()` after writing the helpers; it uses the 14-row lecture example.

Next complete `FrequencyEncoderTransformer` and `ImpactEncoderTransformer`. Each should convert string columns to floats while keeping numeric columns numeric. In `fit`, store a mapping for each categorical column using only the data passed to `fit`. In `transform`, apply the stored mappings and return a 2D NumPy array of floats. For a category absent from the fitted data, use 0.0 for frequency encoding and the training set's overall class-1 proportion for impact encoding. `is_numeric_col` is provided to help identify numeric columns in the mixed-type array.

Use `load_credit_approval_data()` to load the binary credit approval problem. The loader drops rows with missing values so you can focus on encoding. Verify that your transformed data is numeric and that a classifier such as `DecisionTreeClassifier` can fit it. For a train/test comparison, split the original rows first, then fit the transformer on training rows only. Never use test labels to fit the impact encoder.

The credit data comes from the [UCI Statlog Australian Credit Approval dataset](#). If you want to go further, consider multiclass impact encoding or using scikit-learn's `ColumnTransformer` to select particular columns.

Task 2: Regression and regression trees

Open `code/lab3/lab3_task2.py`. It loads the supplied data `lab3/winequality-red.csv`. The `quality` column is the target score; the other 11 columns are numeric features. The starter makes a 70% training and 30% test split with a fixed random seed. Work only with the training data while fitting each model.

Fit `LinearRegression` and print mean squared error (MSE) for its predictions on both the training and test sets. Do the same for `DecisionTreeRegressor(max_depth=3, random_state=42)`. Compare their errors. Try several tree depths. What happens to training and test MSE as the tree grows deeper, and how does this relate to overfitting?

Finally, fit `RandomForestRegressor(random_state=42)` and compare its training and test MSE with the single regression tree. A regression forest averages continuous predictions from its trees. The supplied red-wine data is the dataset discussed in the Week 3 lecture and comes from the [UCI Wine Quality dataset](#).

Task 3: Implement a majority-vote ensemble

Open `code/lab3/lab3_task3.py` and complete `BasicEnsembleClassifier`. Its constructor already stores the parameters required by scikit-learn. If `base_estimator` is `None`, use a `DecisionTreeClassifier(max_depth=5)`. In `fit`, clone that classifier for each ensemble member with `sklearn.base.clone`. Draw a random subset of the training rows for each member. `sample_fraction` defaults to 0.6; `replace` controls sampling with or without replacement. Use `np.random.default_rng(self.random_state)` for reproducibility.

For each member, one way to draw rows is:

```
m = max(1, int(round(self.sample_fraction * len(X))))
selected = rng.choice(len(X), size=m, replace=self.replace)
X_sample, y_sample = X[selected], y[selected]
```

In `predict`, collect every member's class prediction for each row and return the majority vote. If there is a tie, choose the smallest class label. Check the classifier on the credit approval data. Then put your Task 1 transformer and classifier together in a scikit-learn `Pipeline`, and fit and evaluate the pipeline on separate training and test rows.

Optional extension: Compare ensembles

Use a 70/30 train/test split of the credit data. Fit preprocessing on training rows only. Compare your ensemble with `RandomForestClassifier` and `aeon.classification.sklearn.RotationForestClassifier`, recording test accuracy for each. How do `sample_fraction` and `replace` change your ensemble's results? How does its construction differ from a random forest? If time permits, add `predict_proba` by averaging member probabilities and see whether it changes accuracy.